PERTEMUAN 9 · Praktikum 2

Top-Down Parsing: Recursive Descent, LL(1), FIRST, dan FOLLOW

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 03 — Merancang tata bahasa bebas konteks untuk bahasa sederhana; CPMK 04 — Mengimplementasikan parser sederhana untuk validasi kode.

Sub-CPMK yang dinilai

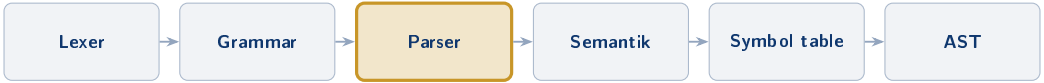
- **Sub-CPMK 3.3:** Membuat tabel himpunan FIRST dan FOLLOW untuk grammar subset bahasa C dan menguji keputusan parser LL(1) berdasarkan tabel tersebut.
- **Sub-CPMK 4.1:** Mengimplementasikan *recursive descent parser* dan *predictive parser* LL(1) dalam bahasa C, dengan berkas kode dan keluaran validasi sebagai bukti.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
20	Briefing Praktikum 2 dan cakupan artefak
25	Ceramah: recursive descent dan predictive parsing
30	Latihan: menghitung FIRST dan FOLLOW
30	Lokakarya: tabel LL(1) dan penelusuran stack
35	Praktik: parser C, AST, dan diagnostik
10	Pengujian, laporan, dan penutup

Porsi praktik dan latihan 105 menit; ceramah 25 menit. Setiap mahasiswa menyerahkan tabel dan parser.

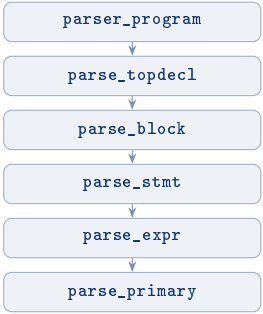
Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini membangun simpul **parser**: dari grammar bersih menjadi pengenal struktur program.
- Keluaran parser berupa AST menjadi masukan analisis semantik pada Pertemuan 13–15.
- Praktikum 2 menilai Sub-CPMK 3.1–3.4 dan 4.1.

Recursive Descent: Satu Fungsi per Nonterminal

- Satu fungsi untuk setiap nonterminal.
- `parseE` mengenali string dari E; `parseT` dari T.
- Setiap fungsi memilih aturan berdasarkan token saat ini.
- Mudah ditulis, mudah di-*debug*, dan pesan kesalahan baik.
- Syarat: tanpa *left recursion*, sudah difaktorkan, satu token lookahead.



Struktur Parser dan Primitifnya

Struktur `Parser`

Menyimpan `Lexer`, token saat ini (`current`), dan status kesalahan (`had_error`).

- `advance()`: mengambil token berikutnya dari lexer.
- `check(type)`: menguji jenis token tanpa mengonsumsinya.
- `match(type)`: mengonsumsi token jika jenisnya sesuai.
- Ketiganya menjadi primitif seluruh parser.

Jika token tidak cocok, parser melaporkan kesalahan dan mengembalikan simpul pengganti agar analisis dapat dilanjutkan.

Contoh Fungsi: parse_primary

```
107 static AstNode *parse_primary(Parser *parser) {
108     Token token = parser->current;
109
110     if (check(parser, TOK_INT_LIT) || check(parser,
          TOK_FLOAT_LIT) ||
111         check(parser, TOK_CHAR_LIT) || check(parser,
              TOK_STRING_LIT)) {
112         AstKind kind = AST_INT_LIT;
113         if (token.type == TOK_FLOAT_LIT) {
114             kind = AST_FLOAT_LIT;
115         } else if (token.type == TOK_CHAR_LIT) {
116             kind = AST_CHAR_LIT;
117         } else if (token.type == TOK_STRING_LIT) {
118             kind = AST_STRING_LIT;
119         }
120         advance(parser);
121         return ast_new(kind, token);
```

Listing 1: Awal parse_primary pada parser.c

- Literal integer, float, char, dan string.
- Identifier, pemanggilan, dan ekspresi dalam kurung ditangani cabang berikutnya.
- Setiap cabang memeriksa token saat ini.
- Tidak ada cabang cocok: kesalahan sintaksis.

Himpunan FIRST

Definisi

FIRST(α) adalah himpunan terminal yang dapat muncul sebagai simbol pertama pada string turunan α. Jika α dapat menurunkan ε, maka $\varepsilon \in$ FIRST(α).

1. Jika X terminal, FIRST$(X) = \{X\}$.
2. Jika $X \to \varepsilon$, tambahkan ε.
3. Jika $X \to Y_1 Y_2 \cdots Y_k$, tambahkan FIRST$(Y_1) \setminus \{\varepsilon\}$; jika Y_1 nullable, lanjut ke Y_2, dan seterusnya.
4. Ulangi sampai tidak ada perubahan.

Himpunan FOLLOW

Definisi

FOLLOW(A) adalah himpunan terminal yang dapat muncul tepat setelah A pada suatu bentuk kalimat. Simbol akhir masukan \$ ditambahkan ke FOLLOW simbol awal.

1. Tambahkan \$ ke FOLLOW(S).
2. Untuk $A \to \alpha B \beta$, tambahkan FIRST(β) $\setminus \{\varepsilon\}$ ke FOLLOW(B).
3. Jika β nullable atau kosong, tambahkan FOLLOW(A) ke FOLLOW(B).
4. Ulangi sampai stabil.

Tabel FIRST dan FOLLOW (Artefak Sub-CPMK 3.3)

Nonterminal	FIRST	FOLLOW
E	$\{\texttt{id}, (\}$	$\{\$,)\}$
E'	$\{+, \varepsilon\}$	$\{\$,)\}$
T	$\{\texttt{id}, (\}$	$\{\$,), +\}$
T'	$\{*, \varepsilon\}$	$\{\$,), +\}$
F	$\{\texttt{id}, (\}$	$\{\$,), *, +\}$

Grammar yang dihitung

$E \to TE'$, $E' \to +TE' \mid \varepsilon$, $T \to FT'$, $T' \to *FT' \mid \varepsilon$, $F \to (E) \mid \texttt{id}$.

Setiap nilai dihitung dari aturan grammar, bukan dari intuisi.

Keputusan LL(1)

Syarat LL(1)

Untuk setiap nonterminal A dan pasangan aturan $A \to \alpha$, $A \to \beta$: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$; jika salah satu nullable, $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing$.

Keadaan grammar	Sel ganda	Keputusan
Grammar ekspresi ini	tidak ada	LL(1)
Grammar dengan *left recursion*	ada	bukan LL(1) sebelum transformasi
Grammar dengan prefiks umum	ada	bukan LL(1) sebelum factoring

Tabel Parsing LL(1)

Konstruksi

Untuk aturan $A \to \alpha$ dan $a \in \text{FIRST}(\alpha)$, isi $M[A, a] = A \to \alpha$. Jika $\varepsilon \in \text{FIRST}(\alpha)$, isi $M[A, b] = A \to \alpha$ untuk setiap $b \in \text{FOLLOW}(A)$. Sel kosong berarti kesalahan sintaksis.

	`id`	`+`	`*`	`$`
E	$E \to TE'$			
E'		$E' \to +TE'$		$E' \to \varepsilon$
T	$T \to FT'$			
T'		$T' \to \varepsilon$	$T' \to *FT'$	$T' \to \varepsilon$
F	$F \to$ `id`			

Kolom (dan) diisi aturan yang sama untuk E, T dan ε untuk E', T'.

Algoritma Predictive Parser

1. Dorong $\$$ dan simbol awal ke stack.
2. a adalah token saat ini.
3. Selama stack tidak kosong:
   3. X adalah puncak stack;
   3. jika X terminal atau $\$$, cocokkan dengan a; jika cocok, pop dan majukan a;
   3. jika X nonterminal, lihat $M[X, a]$;
   3. jika aturan $X \to Y_1 \cdots Y_k$ ada, pop X dan dorong $Y_k, \ldots, Y_1$;
   3. jika sel kosong, laporkan kesalahan.

Parser tidak pernah mundur; waktu linear terhadap panjang masukan.

Penelusuran Stack untuk `id + id * id`

Stack	Masukan	Aksi
$\$E$	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	$E \to TE'$
$\$E'T$	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	$T \to FT'$
$\$E'T'F$	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	$F \to \texttt{id}$
$\$E'T'\texttt{id}$	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	cocokkan `id`
$\$E'T'$	$+\texttt{id} * \texttt{id}\ \$$	$T' \to \varepsilon$
$\$E'$	$+\texttt{id} * \texttt{id}\ \$$	$E' \to +TE'$
$\$E'T+$	$+\texttt{id} * \texttt{id}\ \$$	cocokkan $+$
$\$E'T$	$\texttt{id} * \texttt{id}\ \$$	$T \to FT'$
... penelusuran berlanjut sampai stack hanya berisi $ dan masukan $		
$	$	terima

Dari Tabel ke Fungsi Parser

- Aturan $A \to X_1 \cdots X_k$: panggil `parseX1`, `parseX2`, dan seterusnya.
- Terminal: panggil `match`.
- ε: tidak ada aksi.
- Sel kosong: laporkan kesalahan.

Nonterminal	Fungsi	Tanggung jawab
`program`	`parser_program`	seluruh berkas
`topdecl`	`parse_topdecl`	deklarasi fungsi
`block`	`parse_block`	daftar pernyataan
`stmt`	`parse_stmt`	satu pernyataan
`expr-primary`	`parse_expr`, ...	ekspresi berprioritas

Pemilihan Cabang: parse_stmt

```
296 static AstNode *parse_stmt(Parser *parser) {
297     Token token = parser->current;
298
299     if (is_type(token.type)) {
300         AstNode *node = parse_decl(parser);
301         expect(parser, TOK_SEMI, "diharapkan ';'
                setelah deklarasi");
302         return node;
303     }
304
305     if (check(parser, TOK_LBRACE)) {
306         return parse_block(parser);
307     }
308
309     if (check(parser, TOK_IF)) {
```

Listing 2: Bagian awal parse_stmt pada parser.c

- Tipe memulai deklarasi lokal.
- { memulai blok.
- if memilih parse_if.
- Cabang lain ditangani dengan pola yang sama.
- Tidak ada cabang cocok: kesalahan dan simpul pengganti.

Sinkronisasi Setelah Kesalahan

```
42 static void synchronize(Parser *parser) {
43     while (!check(parser, TOK_EOF) &&
44            !check(parser, TOK_SEMI) &&
45            !check(parser, TOK_RBRACE)) {
46         advance(parser);
47     }
48     if (check(parser, TOK_SEMI)) {
49         advance(parser);
50     }
51 }
52
53 static int is_type(TokenType type) {
54     return type == TOK_INT || type == TOK_FLOAT ||
55            type == TOK_CHAR || type == TOK_VOID;
```

Listing 3: Fungsi synchronize pada parser.c

- Majukan token sampai titik aman.
- Titik aman: ;, }, atau awal pernyataan berikutnya.
- Terlalu agresif: laporan palsu.
- Terlalu konservatif: kesalahan terlewat.

Kesalahan Sintaksis dan Pemulihan

Jenis kesalahan

- titik koma hilang;
- kurung tidak seimbang;
- kata kunci salah;
- ekspresi tidak lengkap;
- deklarasi fungsi tanpa nama.

Strategi pemulihan

1. *panic mode*: lewati token sampai titik sinkronisasi;
2. *phrase-level recovery*: sisipkan token lokal;
3. *error production*: aturan khusus untuk kesalahan umum.

Pesan yang baik memuat lokasi, harapan, temuan, dan saran perbaikan.

Contoh Pesan Kesalahan Sintaksis

Masukan	Pesan
`int a = 1`	`baris 1, kolom 10: diharapkan ';' setelah deklarasi, ditemukan 'return'`
`if (a`	`baris 1, kolom 6: diharapkan ')' setelah kondisi`
`return ;`	`baris 1, kolom 8: ekspresi diharapkan setelah return`
`int (`	`baris 1, kolom 5: nama fungsi diharapkan`

Jika masukan terlalu rusak, parser berhenti setelah sejumlah kesalahan dan melaporkan ringkasan.

Kegiatan Praktikum: M2

Langkah kerja

1. Hitung FIRST dan FOLLOW untuk grammar hasil Pertemuan 7.
2. Bangun tabel LL(1) dan periksa konflik.
3. Implementasikan fungsi parser per nonterminal.
4. Bangun AST saat mengurangi aturan.
5. Tambahkan sinkronisasi dan pesan kesalahan.
6. Uji dengan program valid dan tidak valid.

Kumpulkan juga artefak 3.1, 3.2, dan 3.4 dari Pertemuan 6–7 karena Praktikum 2 menilai 3.1–3.4 dan 4.1.

Rubrik Praktikum 2

Aspek	Indikator	Bobot
Kebenaran	Parser menerima program valid dan menolak yang tidak valid	40%
Kualitas kode	Struktur fungsi, penamaan, komentar, tanpa peringatan	25%
Kelengkapan	Ekspresi, pernyataan, AST, dan diagnostik	20%
Pengujian	Kasus normal, batas, dan kesalahan terdokumentasi	15%

Rubrik ini sama dengan rubrik artefak praktikum pada bagian 10 RPS.

Tugas dan Artefak

Artefak Sub-CPMK 3.3 dan 4.1

Tabel FIRST/FOLLOW, keputusan LL(1), *recursive descent parser*, dan *predictive parser* LL(1) dalam bahasa C, disertai keluaran validasi.

- Tabel FIRST/FOLLOW lengkap dan keputusan LL(1) beralasan.
- Parser C lulus minimal 20 kasus uji valid dan tidak valid.
- AST terbentuk untuk program valid.
- Diagnostik menyebut lokasi dan token yang diharapkan.
- Artefak 3.1, 3.2, dan 3.4 dari Pertemuan 6–7 ikut dikumpulkan.
- Laporan singkat diserahkan.

Praktikum 2 berbobot 12% dan dilaporkan setelah Milestone M2 selesai.

Rangkuman dan Jembatan

- Recursive descent memakai satu fungsi per nonterminal dan memilih aturan berdasarkan token saat ini.
- FIRST menentukan terminal awal suatu string; FOLLOW menentukan terminal setelah nonterminal.
- Grammar LL(1) jika tabel parsing tidak memiliki sel ganda.
- Predictive parser memakai stack eksplisit dan tabel LL(1) tanpa mundur.
- Pemulihan kesalahan memakai *panic mode*, *phrase-level recovery*, atau *error production*.

Pertemuan berikutnya

Pertemuan 10 beralih ke bottom-up parsing: *shift-reduce* dan operator precedence. Pendekatan ini menangani *left recursion* secara alami.